

APOSTILA PROFISSIONAL COMPLETA

HTML5 & CSS3

Do Zero ao Avançado — Edição Estendida e Didática

+ de 100 exemplos práticos

Explicações passo a passo

Para iniciantes

Projeto final incluso

Um material feito para quem está começando do absoluto zero no desenvolvimento web. Cada tag e cada propriedade é explicada com linguagem simples, exemplos completos de código e o resultado esperado, para que você realmente entenda o "porquê" por trás de cada linha.

Conteúdo: Fundamentos da Web, HTML5 completo, CSS3 completo, Flexbox, Grid, Responsividade, Animações e Projeto Prático Final.

Sumário

MÓDULO 1 — Introdução ao Desenvolvimento Web

- Como a internet realmente funciona
- O que é HTML e o que é CSS
- Front-end vs Back-end
- Ferramentas necessárias (editor, navegador)
- Como o navegador lê seu código

MÓDULO 2 — Estrutura Completa do HTML5

- Anatomia de uma tag
- Estrutura mínima de um documento
- Explicação linha a linha do boilerplate
- Atributos globais e comentários

MÓDULO 3 — Trabalhando com Textos

- Títulos e hierarquia (h1–h6)
- Parágrafos e quebras de linha
- Formatações de texto (negrito, itálico, marcação, etc.)
- Citações e código
- Links internos, externos e âncoras

MÓDULO 4 — Listas, Tabelas, Imagens, Áudio e Vídeo

- Listas ordenadas, não ordenadas e de definição
- Tabelas completas (thead, tbody, colspan, rowspan)
- Imagens, atributos e imagens responsivas com <picture>
- Áudio e vídeo embutidos

MÓDULO 5 — Formulários Profissionais

- Estrutura de um formulário (form, action, method)
- Todos os tipos de <input>
- Select, textarea, checkbox, radio
- Validações nativas do HTML5
- Acessibilidade em formulários (label, fieldset)

MÓDULO 6 — Tags Semânticas e Estrutura de Página

- Por que a semântica importa (SEO e acessibilidade)
- header, nav, main, section, article, aside, footer
- Montando o esqueleto de um site completo

MÓDULO 7 — Introdução ao CSS3

- O que é e para que serve o CSS
- As 3 formas de aplicar CSS (inline, interno, externo)
- Sintaxe de uma regra CSS
- Cascata, herança e especificidade

MÓDULO 8 — Seletores CSS Completos

- Seletores básicos (elemento, classe, id, universal)
- Seletores de atributo
- Combinadores (descendente, filho, irmão)
- Pseudo-classes e pseudo-elementos

MÓDULO 9 — Box Model, Cores, Fontes e Espaçamentos

- O Box Model explicado visualmente
- box-sizing: content-box vs border-box
- Formatos de cor (nome, hex, rgb, rgba, hsl)
- Tipografia e unidades de medida (px, %, em, rem, vw, vh)

MÓDULO 10 — Display, Posicionamento e Overflow

- display: block, inline, inline-block, none
- position: static, relative, absolute, fixed, sticky
- z-index e empilhamento
- overflow e controle de conteúdo

MÓDULO 11 — Flexbox Completo

- Conceito de eixo principal e cruzado
- Propriedades do container (display flex, justify-content, align-items...)
- Propriedades dos itens (flex-grow, flex-shrink, order...)
- Exemplos práticos de layout com Flexbox

MÓDULO 12 — CSS Grid Completo

- Diferença entre Grid e Flexbox
- grid-template-columns e rows

- fr, repeat(), minmax(), gap
- grid-template-areas na prática

MÓDULO 13 — Responsividade

- O que é design responsivo e mobile-first
- A tag viewport
- Media queries na prática
- Breakpoints mais usados no mercado

MÓDULO 14 — Efeitos Visuais e Animações

- Hover, focus e transições suaves
- Transform (translate, scale, rotate, skew)
- Animações com @keyframes
- Sombras e gradientes

MÓDULO 15 — Variáveis CSS, Boas Práticas e Projeto Final

- Variáveis CSS (custom properties)
- Organização de código e boas práticas
- Projeto prático: Landing Page completa (HTML + CSS comentados)

Introdução ao Desenvolvimento Web

Antes de escrever a primeira linha de código, é fundamental entender **o que** estamos construindo e **como** tudo funciona por trás dos panos. Este módulo é a base de tudo — sem ele, HTML e CSS parecem "decoreba"; com ele, cada tag e cada propriedade fazem sentido.

Como a internet funciona (na prática)

Toda vez que você digita um endereço no navegador (como `www.google.com`) e aperta Enter, uma sequência de eventos acontece em frações de segundo:

1. **Requisição:** o seu navegador (o "cliente") envia um pedido para um computador remoto chamado **servidor**, perguntando "me envie os arquivos desse site".
2. **Resposta do servidor:** o servidor localiza os arquivos daquele site (HTML, CSS, JavaScript, imagens) e os envia de volta pela internet.
3. **Interpretação (rendering):** o navegador recebe esses arquivos e os "traduz" visualmente para você. Ele lê o HTML para montar a estrutura, aplica o CSS para deixar bonito e executa o JavaScript para dar interatividade.

Analogia simples

Pense em um restaurante: você (cliente) faz um pedido ao garçom (requisição). A cozinha (servidor) prepara o prato e o garçom traz de volta (resposta). O prato pronto na sua mesa é a página carregada no navegador.

O que é HTML?

HTML significa *HyperText Markup Language*, ou "Linguagem de Marcação de Hipertexto". Não é uma linguagem de programação (não existe lógica, condição ou cálculo em HTML puro) — é uma **linguagem de marcação**, ou seja, ela serve para descrever e organizar o conteúdo de uma página.

Com HTML você diz ao navegador coisas como: "isto aqui é um título", "isto é um parágrafo", "isto é uma imagem", "isto é um botão". O HTML é o **esqueleto** do site:

define o que existe e em qual ordem, mas não define aparência.

O que é CSS?

CSS significa *Cascading Style Sheets*, ou "Folhas de Estilo em Cascata". Se o HTML é o esqueleto, o CSS é a **pele, roupa e maquiagem** do site. É ele quem define:

- Cores de fundo e de texto;
- Tamanhos, margens e espaçamentos;
- Fontes e tipografia;
- Posicionamento dos elementos na tela;
- Efeitos visuais, animações e responsividade (adaptação para celular, tablet, etc).

✅ Dica de ouro

Nunca use HTML para "fazer parecer bonito" (como usar `<center>` ou atributos de cor direto na tag). Estrutura é responsabilidade do HTML. Aparência é responsabilidade do CSS. Separar essas duas coisas é a regra mais importante de todo o desenvolvimento web moderno.

Front-end vs Back-end

Front-end	Back-end
É a parte visual , tudo que o usuário vê e interage diretamente no navegador.	É a parte lógica , que roda no servidor e o usuário não vê diretamente.
Tecnologias: HTML, CSS, JavaScript, React, Vue...	Tecnologias: PHP, Node.js, Python, Java, bancos de dados...
Exemplos: layout da página, botões, animações, formulários visuais.	Exemplos: login, cadastro no banco de dados, autenticação, regras de negócio.

Ferramentas que você precisa para começar

- **Editor de código:** recomendado o Visual Studio Code (gratuito), com a extensão "Live Server" para visualizar suas páginas em tempo real.

- **Navegador:** Google Chrome ou Firefox, de preferência com o DevTools (F12) aberto para inspecionar elementos.
- **Não precisa instalar nada além disso** para começar com HTML e CSS — diferente de outras linguagens, eles não precisam de compilador.



Exercício de fixação — Módulo 1

1. Explique com suas próprias palavras a diferença entre HTML e CSS.
2. O que acontece, passo a passo, quando você acessa um site pelo navegador?
3. Cite 2 exemplos de tecnologias de front-end e 2 de back-end.

Estrutura Completa do HTML5

Anatomia de uma tag HTML

Antes de tudo, vamos entender como uma tag é formada. A maioria das tags HTML tem uma abertura e um fechamento, com o conteúdo no meio:

EXEMPLE

```
<p>Este é o conteúdo dentro da tag.</p>
```

↑ ↑

tag de abertura tag de fechamento

Algumas tags não têm conteúdo e não precisam de fechamento — são chamadas de **tags vazias** ou *self-closing*, como `<br>`, `<img>` e `<input>`.

Uma tag também pode receber **atributos**, que são informações extras escritas dentro da tag de abertura, no formato `atributo="valor"` :

```
<a href="https://google.com" target="_blank">Ir para o Google</a>
```

Aqui, `href` e `target` são atributos que dão instruções extras à tag `<a>`.

Estrutura mínima de um documento HTML5

Todo archivo HTML válido seque este esqueleto básico:

BOILERPLATE HTML5

```
<!DOCTYPE html>
<html lang="pt-br">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Meu Primeiro Site</title>
</head>

<body>
  <h1>Olá, mundo!</h1>
</body>

</html>
```

Explicação linha a linha

<!DOCTYPE html>

Não é bem uma tag — é uma **declaração** que fica sempre na primeira linha do arquivo. Ela avisa ao navegador: "este documento foi escrito seguindo as regras do HTML5". Sem ela, navegadores antigos podem interpretar seu código em "modo de compatibilidade", quebrando o layout.

<html lang="pt-br">

É a tag "mãe" de todo o documento — tudo que existe na página fica dentro dela. O atributo `lang="pt-br"` informa o idioma principal do conteúdo. Isso ajuda:

- Leitores de tela (acessibilidade) a pronunciar o texto corretamente;
- Buscadores como o Google a indexar o site para o público certo;
- Navegadores a sugerir traduções automáticas corretamente.

<head>

Contém metadados: informações que **não aparecem visualmente** na página, mas que são essenciais para o funcionamento e para os buscadores. Dentro dele geralmente colocamos:

- `<meta charset="UTF-8">` — define a codificação de caracteres, garantindo que acentos e caracteres especiais (ç, ã, á) apareçam corretamente.
- `<meta name="viewport" ...>` — essencial para responsividade, avisa ao navegador para usar a largura real do dispositivo (sem isso, sites ficam "zoom out")

em celulares).

- `<title>` — o texto que aparece na aba do navegador e no resultado de busca do Google.
- `<link>` — usado para importar arquivos externos, como uma folha de CSS.
- `<meta name="description">` — o resumo do site que aparece no Google.

⚠ Erro comum

Esquecer a meta tag de viewport é o motivo número 1 de sites "quebrados" no celular. Sempre inclua: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

`<body>`

É onde fica todo o **conteúdo visível** da página: textos, imagens, botões, formulários, vídeos, tudo que o usuário realmente vê e interage. Cada site tem exatamente um

`<body>` .

Atributos globais mais usados

Alguns atributos podem ser usados em **quase qualquer** tag HTML:

Atributo	Para que serve
<code>id</code>	Identificador único de um elemento (não pode se repetir na página). Usado para CSS específico e âncoras.
<code>class</code>	Classifica elementos em grupos que compartilham o mesmo estilo. Pode se repetir e um elemento pode ter várias classes.
<code>style</code>	Aplica CSS diretamente na tag (uso desaconselhado em projetos reais).
<code>title</code>	Cria uma dica (tooltip) que aparece ao passar o mouse.
<code>data-*</code>	Cria atributos personalizados para guardar dados que o JavaScript pode ler depois, ex: <code>data-id="102"</code> .

Comentários em HTML

Comentários não aparecem na tela e servem para você (ou sua equipe) deixar anotações no código:

```
<!-- Isso é um comentário, o navegador ignora esta linha -->  
<h1>Título visível</h1>
```



Exercício de fixação — Módulo 2

1. Monte, do zero, um arquivo HTML5 válido contendo `<!DOCTYPE>`, `head` e `body`.
2. Adicione um título de aba personalizado usando `<title>`.
3. Explique por que a meta tag `viewport` é importante para dispositivos móveis.

Trabalhando com Textos

Títulos e hierarquia (h1 a h6)

O HTML oferece 6 níveis de títulos, do mais importante (`h1`) ao menos importante (`h6`). Eles não servem apenas para deixar o texto grande — servem para criar uma **hierarquia lógica** de conteúdo, algo essencial para SEO e acessibilidade.

```
<h1>Título Principal da Página</h1>
<h2>Subtítulo de uma Seção</h2>
<h3>Um tópico dentro da seção</h3>
<h4>Um subtópico</h4>
```

✓ Regra de hierarquia

Use **apenas um** `<h1>` por página (geralmente o título principal do conteúdo). Depois disso, desça a hierarquia gradualmente: um `<h3>` deve estar dentro do contexto de um `<h2>`, nunca pule níveis "porque ficou visualmente maior ou menor" — se você quer só mudar o tamanho da fonte, isso é trabalho do CSS, não da tag.

Parágrafos

A tag `<p>` define um parágrafo de texto. Navegadores adicionam automaticamente um espaço (margem) antes e depois de cada parágrafo.

```
<p>Este é o meu primeiro parágrafo de texto em HTML.</p>
<p>Este é outro parágrafo, completamente separado do anterior.</p>
```

Quebra de linha e linha horizontal

`<br>` — quebra de linha

Força uma quebra de linha **dentro** do mesmo bloco de texto, sem criar um novo parágrafo. Use com moderação — nunca use vários `<br>` seguidos apenas para criar espaçamento (isso é papel do CSS, com `margin`).

```
<p>Rua das Flores, 123<br>Bairro Centro<br>Cidade - Estado</p>
```

<hr> – linha horizontal

Cria uma linha divisória horizontal, geralmente usada para separar seções de conteúdo diferentes.

```
<p>Fim da seção 1</p>
<hr>
<p>Início da seção 2</p>
```

Formatações de texto

Tag	Efeito	Quando usar
<code></code>	Negrito com importância semântica	Para destacar algo realmente importante (leitores de tela dão ênfase na voz)
<code></code>	Negrito visual, sem importância semântica	Apenas estilo, sem significado extra
<code></code>	Itálico com <i>ênfase</i> na entonação	Quando o significado da frase muda com a ênfase
<code><i></code>	Itálico visual (ex: termos técnicos, estrangeiros)	Apenas estilo, sem ênfase de leitura
<code><u></code>	Sublinhado	Uso raro (sublinhado remete a link, pode confundir usuários)
<code><mark></code>	Marca-texto (fundo amarelo por padrão)	Destacar um trecho relevante em uma busca ou resultado
<code><small></code>	Texto menor	Notas de rodapé, termos legais
<code></code>	Texto riscado	Indicar algo removido, ex: preço antigo
<code><ins></code>	Texto sublinhado (inserido)	Indicar algo adicionado recentemente
<code><sub></code> / <code><sup></code>	Subscrito / Sobrescrito	Fórmulas químicas (H ₂ O), expoentes (x ²)

EXEMPLO COMBINANDO FORMATAÇÕES

```
<p>
  <strong>Atenção:</strong> o preço era
  <del>R$ 199,90</del>
  e agora é <mark>R$ 149,90</mark>!
</p>
```

Citações

```
<blockquote>
  A simplicidade é o último grau de sofisticação.
</blockquote>

<p>Segundo o autor, <q>o código limpo é sempre mais rápido de manter</q>.
</p>
```

`<blockquote>` é usada para citações longas (bloco separado), enquanto `<q>` é usada para citações curtas dentro do próprio texto (o navegador adiciona aspas automaticamente).

Código-fonte no HTML

```
<p>Para imprimir na tela, use o comando <code>console.log()</code>.</p>
<pre>
  Este texto preserva
    espaços e
      quebras de linha exatamente como digitado.
</pre>
```

Links

A tag `<a>` (de "anchor" / âncora) cria links de navegação.

Link externo

```
<a href="https://www.google.com">Visitar o Google</a>
```

Abrir em nova aba

```
<a href="https://www.google.com" target="_blank" rel="noopener
noreferrer">
  Abrir em nova aba
</a>
```

💡 Por que usar `rel="noopener noreferrer"`?

Quando você abre um link em nova aba com `target="_blank"`, a nova página pode, por falha de segurança, obter acesso à página de origem. O atributo `rel="noopener noreferrer"` bloqueia esse acesso — é uma boa prática de segurança sempre usada junto com `target="_blank"`.

Link para outra página do mesmo site (relativo)

```
<a href="sobre.html">Sobre nós</a>
<a href="contato.html">Fale conosco</a>
```

Link de âncora (rolar para uma parte da mesma página)

```
<a href="#contato">Ir para contato</a>

...

<section id="contato">
  <h2>Fale Conosco</h2>
</section>
```

Link de e-mail e telefone

```
<a href="mailto:contato@site.com">Enviar e-mail</a>
<a href="tel:+5511999999999">Ligar agora</a>
```



Exercício de fixação — Módulo 3

1. Crie uma página com h1, h2 e três parágrafos sobre um assunto de sua escolha.
2. Destaque uma palavra importante com `<strong>` e outra com `<em>`.
3. Crie 3 links: um externo (abrindo em nova aba), um de e-mail e um de âncora para outra parte da página.

Listas, Tabelas, Imagens, Áudio e Vídeo

Listas não ordenadas

Usadas quando a ordem dos itens não importa. Cada item vem marcado com um "bullet" (•) por padrão.

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

Listas ordenadas

Usadas quando a sequência importa (passo a passo, ranking, instruções).

```
<ol>
  <li>Abrir o editor de código</li>
  <li>Criar o arquivo index.html</li>
  <li>Escrever a estrutura básica</li>
</ol>
```

É possível controlar o início da contagem com o atributo `start`, e inverter com `reversed`: `<ol start="5" reversed>`.

Listas de definição

Menos comum, usada para pares de termo/descrição, como um glossário:

```
<dl>
  <dt>HTML</dt>
  <dd>Linguagem de marcação usada para estruturar páginas.</dd>

  <dt>CSS</dt>
  <dd>Linguagem usada para estilizar páginas.</dd>
</dl>
```

Listas aninhadas

Uma lista pode conter outra lista dentro de um `<li>`, criando submenus ou sub-tópicos:

```
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
  <li>Backend</li>
</ul>
```

Tabelas completas

Tabelas devem ser usadas **apenas para dados tabulares** (planilhas, listas de preços, comparações) — nunca para criar o layout de uma página, isso é papel do CSS (Flexbox/Grid).

```

<table>
  <thead>
    <tr>
      <th>Produto</th>
      <th>Preço</th>
      <th>Estoque</th>
    </tr>
  </thead>

  <tbody>
    <tr>
      <td>Notebook</td>
      <td>R$ 3.200</td>
      <td>12</td>
    </tr>
    <tr>
      <td>Mouse</td>
      <td>R$ 60</td>
      <td>45</td>
    </tr>
  </tbody>
</table>

```

Tag	Função
<table>	Elemento raiz da tabela
<thead>	Agrupa o cabeçalho da tabela
<tbody>	Agrupa o corpo (dados) da tabela
<tfoot>	Agrupa o rodapé (ex: totais)
<tr>	Table Row — uma linha da tabela
<th>	Table Header — célula de cabeçalho (negrito e centralizada por padrão)
<td>	Table Data — célula de dado comum

Mesclando células com colspan e rowspan

```

<tr>
  <td colspan="2">Ocupa 2 colunas</td>
</tr>

```

`colspan` mescla células na horizontal (várias colunas), enquanto `rowspan` mescla na vertical (várias linhas).

Imagens

```

```

Atributo	Explicação
<code>src</code>	Caminho (local ou URL) do arquivo de imagem
<code>alt</code>	Texto alternativo exibido caso a imagem não carregue. Essencial para acessibilidade (leitores de tela leem esse texto) e para SEO.
<code>width</code> / <code>height</code>	Dimensões da imagem — definir ambos ajuda o navegador a reservar o espaço antes da imagem carregar, evitando "pulos" de layout.
<code>loading="lazy"</code>	Adia o carregamento da imagem até que ela esteja próxima da área visível, melhorando a performance da página.

⚠ Nunca deixe o alt vazio sem motivo

Um `alt=""` vazio só deve ser usado em imagens puramente decorativas. Para imagens com conteúdo relevante, sempre descreva o que ela mostra, como

```
alt="Gráfico de vendas crescendo em 2024" .
```

Imagens responsivas com <picture>

Permite servir imagens diferentes dependendo do tamanho da tela, economizando dados em dispositivos móveis:

```
<picture>
  <source media="(max-width: 600px)" srcset="foto-mobile.jpg">
  <source media="(min-width: 601px)" srcset="foto-desktop.jpg">
  
</picture>
```

Áudio

```
<audio controls>
  <source src="musica.mp3" type="audio/mpeg">
  Seu navegador não suporta áudio.
</audio>
```

O atributo `controls` exibe os botões de play, pausa e volume. É possível usar `autoplay` e `loop`, mas use com cautela — reprodução automática de som costuma incomodar o usuário.

Vídeo

```
<video controls width="640" poster="capa.jpg">
  <source src="video.mp4" type="video/mp4">
  Seu navegador não suporta vídeo.
</video>
```

`poster` define uma imagem de capa exibida antes do vídeo começar a rodar.



Exercício de fixação — Módulo 4

1. Crie uma tabela com 3 colunas e 4 linhas mostrando um cardápio (item, descrição, preço).
2. Insira uma imagem com atributo alt descritivo e width/height definidos.
3. Crie uma lista não ordenada com uma sublista aninhada dentro de um dos itens.

Formulários Profissionais

Estrutura de um formulário

```
<form action="/enviar" method="POST">
  ... campos aqui ...
  <button type="submit">Enviar</button>
</form>
```

Atributo	Explicação
<code>action</code>	Para onde os dados do formulário serão enviados (uma URL/rota do back-end)
<code>method</code>	<code>GET</code> envia dados visíveis na URL (bom para buscas); <code>POST</code> envia dados de forma oculta no corpo da requisição (bom para senhas, cadastros)

Label — essencial para acessibilidade

Todo campo de formulário deveria ter um `<label>` associado. Isso permite que, ao clicar no texto do label, o campo seja automaticamente focado — e é essencial para leitores de tela.

```
<label for="email">Seu e-mail:</label>
<input type="email" id="email" name="email">
```

✅ Dica

O valor do `for` no label deve ser **idêntico** ao `id` do input. É essa ligação que ativa o comportamento de clique.

Todos os tipos de `<input>`

Tipo	Uso
<code>text</code>	Texto simples de uma linha

<code>password</code>	Ocultar os caracteres digitados
<code>email</code>	Valida automaticamente formato de e-mail
<code>number</code>	Aceita apenas números, com setas de incremento
<code>tel</code>	Campo de telefone (abre teclado numérico no celular)
<code>url</code>	Valida formato de URL
<code>date</code>	Seletor de data nativo
<code>time</code>	Seletor de horário
<code>checkbox</code>	Caixa de marcação (múltipla escolha)
<code>radio</code>	Botão de opção única (escolha exclusiva dentro do mesmo <code>name</code>)
<code>file</code>	Upload de arquivos
<code>range</code>	Controle deslizante (slider)
<code>color</code>	Seletor de cor visual
<code>hidden</code>	Campo invisível, usado para enviar dados sem mostrar ao usuário
<code>search</code>	Campo de busca (com "x" para limpar)
<code>submit</code>	Botão que envia o formulário

EXEMPLO — CHECKBOX E RADIO

```
<label><input type="checkbox" name="aceite"> Aceito os termos</label>

<label><input type="radio" name="genero" value="m"> Masculino</label>
<label><input type="radio" name="genero" value="f"> Feminino</label>
```

Note que os dois radios têm o **mesmo** `name="genero"` — isso é o que faz o navegador tratá-los como opções mutuamente exclusivas (marcar um desmarca o outro).

Select (caixa de seleção)

```
<label for="estado">Estado:</label>
<select id="estado" name="estado">
  <option value="">Selecione...</option>
  <option value="sp">São Paulo</option>
  <option value="rj">Rio de Janeiro</option>
</select>
```

Textarea (texto de múltiplas linhas)

```
<label for="msg">Mensagem:</label>
<textarea id="msg" name="msg" rows="5" placeholder="Digite aqui...">
</textarea>
```

Validações nativas do HTML5

O HTML5 permite validar campos sem precisar de nenhum JavaScript:

Atributo	Efeito
<code>required</code>	Campo obrigatório, impede envio se vazio
<code>minlength</code> / <code>maxlength</code>	Define quantidade mínima/máxima de caracteres
<code>min</code> / <code>max</code>	Valor mínimo/máximo (para number, date, range)
<code>pattern</code>	Valida o valor contra uma expressão regular
<code>placeholder</code>	Texto de exemplo exibido enquanto o campo está vazio
<code>disabled</code>	Desabilita o campo (não pode ser editado nem enviado)
<code>readonly</code>	Campo não editável, mas o valor é enviado no formulário

EXEMPLO COM VALIDAÇÕES

```
<input
  type="text"
  name="usuario"
  required
  minlength="3"
  maxlength="20"
  placeholder="Digite seu usuário"
>
```

Agrupando campos com fieldset e legend

```
<fieldset>
  <legend>Dados Pessoais</legend>

  <label for="nome">Nome:</label>
  <input type="text" id="nome" name="nome">
</fieldset>
```

`<fieldset>` agrupa visualmente campos relacionados (desenha uma borda ao redor), e `<legend>` é o título desse grupo.



Exercício de fixação — Módulo 5

1. Monte um formulário de cadastro com: nome (required), e-mail (type email), senha (type password, minlength 6), data de nascimento e um select de estado.
2. Adicione um grupo de radio buttons para "gênero" e um checkbox de "aceito os termos".
3. Agrupe os campos usando fieldset e legend.

Tags Semânticas e Estrutura de Página

Por que a semântica importa?

Antes do HTML5, era comum ver páginas inteiras construídas apenas com `<div>` e `<span>`, sem nenhum significado. O problema é que isso deixa o código "cego" para máquinas: buscadores e leitores de tela não conseguem entender qual parte é o menu, qual é o conteúdo principal e qual é o rodapé.

As **tags semânticas** resolvem isso: elas descrevem o **papel** de cada bloco na página, trazendo benefícios diretos de:

- **SEO** — o Google entende melhor a estrutura e prioriza o conteúdo relevante;
- **Acessibilidade** — leitores de tela conseguem "pular" direto para o menu, conteúdo principal, etc;
- **Manutenção** — outros desenvolvedores entendem seu código mais rápido.

As principais tags semânticas

Tag	Representa
<code><header></code>	Cabeçalho da página ou de uma seção (geralmente contém logo, título e menu)
<code><nav></code>	Bloco de navegação (menu principal, breadcrumbs)
<code><main></code>	Conteúdo principal e único da página (usado apenas uma vez)
<code><section></code>	Uma seção temática de conteúdo, geralmente com seu próprio título
<code><article></code>	Conteúdo independente e "auto-suficiente" (um post de blog, uma notícia, um card de produto)
<code><aside></code>	Conteúdo relacionado, mas secundário (barra lateral, anúncios, links relacionados)
<code><footer></code>	Rodapé da página ou de uma seção (créditos, contato, links legais)

💡 section vs article vs div

Use `<article>` quando o conteúdo faz sentido "sozinho" fora daquele contexto (ex: um post que poderia ser compartilhado independentemente). Use `<section>` para agrupar conteúdo relacionado dentro da página (ex: "Sobre nós", "Serviços"). Use `<div>` apenas quando não existe nenhum significado semântico — é só uma "caixa" para fins de estilização com CSS.

Montando o esqueleto completo de um site

```
<body>

  <header>
    <h1>Minha Empresa</h1>
    <nav>
      <ul>
        <li><a href="#inicio">Início</a></li>
        <li><a href="#servicos">Serviços</a></li>
        <li><a href="#contato">Contato</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="servicos">
      <h2>Nossos Serviços</h2>

      <article>
        <h3>Desenvolvimento Web</h3>
        <p>Criamos sites profissionais e responsivos.</p>
      </article>

      <article>
        <h3>Design UI/UX</h3>
        <p>Interfaces bonitas e fáceis de usar.</p>
      </article>
    </section>

    <aside>
      <h4>Você sabia?</h4>
      <p>Trabalhamos com tecnologias modernas desde 2018.</p>
    </aside>
  </main>

  <footer>
    <p>&copy; 2026 Minha Empresa. Todos os direitos reservados.</p>
  </footer>

</body>
```

Repare como, mesmo sem nenhuma cor ou estilo, só de ler as tags já entendemos perfeitamente a estrutura da página: existe um cabeçalho com menu, um conteúdo

principal com seções e artigos, uma barra lateral e um rodapé. **Essa é a essência do HTML bem escrito.**



Exercício de fixação — Módulo 6

1. Recrie o esqueleto acima adicionando uma seção extra "Sobre nós" com um article dentro.
2. Explique, com suas palavras, quando usar `<article>` em vez de `<section>`.
3. Adicione um `<aside>` com uma lista de "posts relacionados".

Introdução ao CSS3

As 3 formas de aplicar CSS

1. CSS inline (dentro da própria tag)

```
<p style="color: red; font-size: 20px;">Texto vermelho</p>
```

Evite ao máximo. Mistura estrutura com estilo, é difícil de manter e tem a maior especificidade (dificulta sobrescrever depois).

2. CSS interno (dentro do <head>)

```
<head>
  <style>
    p {
      color: red;
      font-size: 20px;
    }
  </style>
</head>
```

Útil para testes rápidos ou páginas únicas, mas não escala bem em projetos com várias páginas.

3. CSS externo (arquivo separado) – o método profissional

```
// arquivo: index.html
<head>
  <link rel="stylesheet" href="style.css">
</head>

// arquivo: style.css
p {
  color: red;
  font-size: 20px;
}
```

✅ Padrão do mercado

Sempre use CSS externo em projetos reais. Isso permite reutilizar o mesmo arquivo de estilo em várias páginas do site, além de deixar o HTML limpo e focado apenas na estrutura.

Sintaxe de uma regra CSS

```
seletor {  
  propriedade: valor;  
  propriedade: valor;  
}
```

```
h1 {  
  color: blue;  
  text-align: center;  
}
```

O **seletor** escolhe quais elementos da página serão afetados. Cada **declaração** (propriedade + valor) fica dentro de chaves `{ }` e termina com ponto e vírgula `;`.

Cascata, herança e especificidade

O nome "Cascading" (em cascata) vem do fato de que várias regras podem se aplicar ao mesmo elemento, e o navegador precisa decidir qual "vence". Isso é decidido por 3 fatores, em ordem de prioridade:

1. **Importância:** uma regra com `!important` sempre vence (evite usar isso, é sinal de problema de organização).
2. **Especificidade:** seletores mais "específicos" vencem. Um ID (`#menu`) é mais específico que uma classe (`.menu`), que por sua vez é mais específica que uma tag (`div`).
3. **Ordem no código:** se a especificidade é igual, a última regra declarada no arquivo vence.

Herança

Algumas propriedades CSS (geralmente as de texto, como `color` e `font-family`) são **herdadas** automaticamente pelos elementos filhos. Já propriedades de caixa, como `margin`, `padding` e `border`, **não** são herdadas.

```
body {  
  color: #333; /* todos os textos filhos herdam essa cor */  
  font-family: Arial, sans-serif;  
}
```



Exercício de fixação — Módulo 7

1. Crie um arquivo `style.css` e conecte-o a um HTML usando `<link>`.
2. Escreva uma regra que deixe todos os parágrafos com cor cinza e tamanho 16px.
3. Explique por que CSS externo é preferível ao CSS inline em projetos profissionais.

Seletores CSS Completos

Seletores básicos

Seletor	Exemplo	Seleciona
Universal	<code>*</code>	Todos os elementos da página
Elemento (tag)	<code>p</code>	Todos os <code><p></code>
Classe	<code>.destaque</code>	Todos com <code>class="destaque"</code>
ID	<code>#menu</code>	O único elemento com <code>id="menu"</code>
Agrupado	<code>h1, h2, p</code>	Aplica o mesmo estilo a vários seletores de uma vez

✅ Classe ou ID?

Use **classes** na grande maioria dos casos — elas são reutilizáveis em vários elementos. Reserve **IDs** para casos únicos na página (como âncoras de navegação), já que um ID não deve se repetir e tem alta especificidade, dificultando sobrescrever o estilo depois.

Seletores de atributo

```
input[type="email"] {  
  border: 2px solid blue;  
}  
  
a[target="_blank"] {  
  color: green;  
}
```

Combinadores

Combinador	Exemplo	Significado
------------	---------	-------------

Descendente (espaço)	<code>nav a</code>	Todo <a> dentro de <nav>, em qualquer nível
Filho direto >	<code>ul > li</code>	Apenas que são filhos diretos de
Irmão adjacente +	<code>h2 + p</code>	O <p> imediatamente após um <h2>
Irmão geral ~	<code>h2 ~ p</code>	Todos os <p> que vêm depois de um <h2>, no mesmo nível

Pseudo-classes mais usadas

Pseudo-classe	Quando é ativada
<code>:hover</code>	Quando o mouse passa sobre o elemento
<code>:focus</code>	Quando um campo está selecionado/focado
<code>:active</code>	No momento do clique
<code>:visited</code>	Link já visitado
<code>:first-child</code>	Elemento que é o primeiro filho do seu pai
<code>:last-child</code>	Elemento que é o último filho do seu pai
<code>:nth-child(n)</code>	Elemento na posição "n" (aceita fórmulas como <code>2n</code> para pares)
<code>:not(selector)</code>	Seleciona tudo, exceto o que casa com o seletor informado
<code>:checked</code>	Checkbox ou radio marcado
<code>:disabled</code>	Campo desabilitado

EXEMPLO — ZEBRA STRIPING EM TABELA

```
tr:nth-child(even) {
  background-color: #f2f2f2;
}
```

Pseudo-elementos

Pseudo-elemento	Função
-----------------	--------

<code>::before</code>	Insere conteúdo antes do elemento (via propriedade <code>content</code>)
<code>::after</code>	Insere conteúdo depois do elemento
<code>::first-line</code>	Estiliza apenas a primeira linha de um texto
<code>::first-letter</code>	Estiliza apenas a primeira letra (efeito de "capitular")
<code>::placeholder</code>	Estiliza o texto de placeholder de um input

EXEMPLO — ÍCONE ANTES DE UM LINK

```
a.externo::after {
  content: " ↗";
}
```



Exercício de fixação — Módulo 8

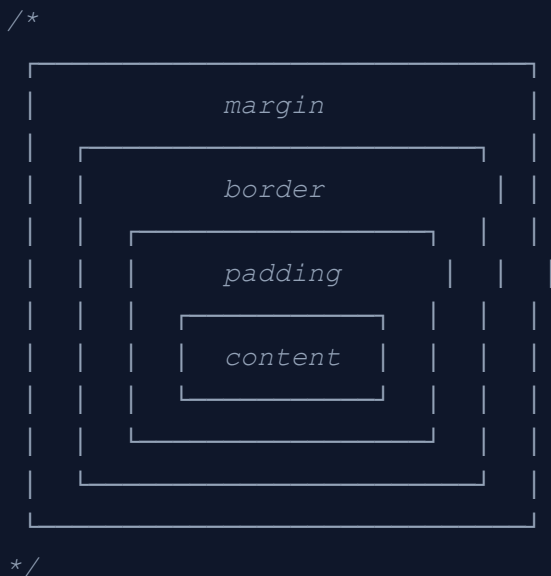
1. Escreva um seletor que aplique estilo apenas aos `<li>` filhos diretos de uma `<ul>` com classe "menu".
2. Crie um efeito de hover que mude a cor de fundo de um botão.
3. Use `:nth-child` para colorir linhas pares de uma tabela.

Box Model, Cores, Fontes e Espaçamentos

O Box Model (Modelo de Caixa)

Todo elemento HTML é, para o CSS, uma **caixa retangular**. Entender o box model é a base de todo o posicionamento em CSS. A caixa é formada por 4 camadas, de dentro para fora:

1. **content** — o conteúdo em si (texto, imagem)
2. **padding** — o espaçamento interno, entre o conteúdo e a borda
3. **border** — a borda da caixa
4. **margin** — o espaçamento externo, entre esta caixa e as caixas vizinhas



```
.caixa {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 10px;  
}
```

box-sizing: content-box vs border-box

Por padrão (`content-box`), quando você define `width: 300px`, o navegador soma o padding e a borda **por fora** desse valor — ou seja, a caixa final fica maior que 300px. Isso confunde muitos iniciantes.

```
* {  
  box-sizing: border-box;  
}
```

Com `border-box`, o padding e a borda passam a ser **incluídos** dentro dos 300px definidos, tornando o dimensionamento muito mais previsível.

✓ Prática padrão do mercado

Quase todo projeto profissional começa com a regra `* { box-sizing: border-box; }` logo no início do CSS. Adote esse hábito desde já.

Margin e Padding — atalhos (shorthand)

Sintaxe	Significado
<code>margin: 10px;</code>	10px em todos os 4 lados
<code>margin: 10px 20px;</code>	10px em cima/baixo, 20px nas laterais
<code>margin: 10px 20px 30px;</code>	10px topo, 20px laterais, 30px embaixo
<code>margin: 10px 20px 30px 40px;</code>	topo, direita, baixo, esquerda (sentido horário)
<code>margin: 0 auto;</code>	Centraliza horizontalmente um bloco com largura definida

Cores em CSS

Formato	Exemplo	Explicação
Nome	<code>color: red;</code>	Palavras-chave predefinidas (limitadas, ~140 nomes)
Hexadecimal	<code>color: #ff0000;</code>	Base 16, formato #RRGGBB (mais usado no mercado)

RGB	<code>color: rgb(255, 0, 0);</code>	Valores de 0 a 255 para vermelho, verde e azul
RGBA	<code>color: rgba(255, 0, 0, 0.5);</code>	Como RGB, com canal alfa (transparência) de 0 a 1
HSL	<code>color: hsl(0, 100%, 50%);</code>	Matiz, Saturação e Luminosidade — mais intuitivo para criar variações de uma mesma cor

Tipografia

```
body {
  font-family: 'Segoe UI', Arial, sans-serif;
  font-size: 16px;
  font-weight: 400;
  line-height: 1.6;
  text-align: left;
  letter-spacing: 0.5px;
  text-transform: uppercase;
}
```

Propriedade	Função
<code>font-family</code>	Define a fonte. Sempre coloque opções alternativas (fallback) caso a primeira não esteja disponível.
<code>font-weight</code>	Peso da fonte: 400 (normal), 700 (negrito), até 900 (extra bold)
<code>line-height</code>	Altura da linha — melhora muito a legibilidade de parágrafos longos (ideal entre 1.4 e 1.8)
<code>text-align</code>	Alinhamento: left, right, center, justify

Unidades de medida

Unidade	Tipo	Explicação
<code>px</code>	Absoluta	Pixel fixo, não escala com configurações do usuário
<code>%</code>	Relativa ao pai	Percentual do elemento pai

<code>em</code>	Relativa ao elemento/pai	Múltiplo do font-size do elemento pai (pode se acumular em cascata)
<code>rem</code>	Relativa à raiz	Múltiplo do font-size do <code><html></code> — mais previsível que <code>em</code> , o preferido para tipografia responsiva
<code>vw</code> / <code>vh</code>	Relativa à viewport	1vw = 1% da largura da tela; 1vh = 1% da altura da tela



Quando usar cada unidade

Use `rem` para fontes e espaçamentos (escala junto com as preferências de acessibilidade do usuário), `%` para larguras fluidas dentro de containers, e `px` para detalhes que realmente precisam ser fixos, como bordas finas de 1px.



Exercício de fixação — Módulo 9

1. Desenhe (no papel ou mentalmente) o box model de uma caixa com padding 20px, border 5px e margin 10px.
2. Explique a diferença prática entre box-sizing content-box e border-box.
3. Converta a cor "azul" (`#0000FF`) para os formatos `rgb()` e `rgba()` com 50% de transparência.

Display, Posicionamento e Overflow

A propriedade display

Valor	Comportamento
<code>block</code>	Ocupa toda a largura disponível e sempre começa em uma nova linha. Aceita <code>width</code> , <code>height</code> , <code>margin</code> e <code>padding</code> em todas as direções. Ex: <code>div</code> , <code>p</code> , <code>h1</code> .
<code>inline</code>	Ocupa apenas o espaço do seu conteúdo, fica lado a lado com outros elementos. Não aceita <code>width/height</code> , e <code>margin/padding</code> vertical é ignorado. Ex: <code>span</code> , <code>a</code> , <code>strong</code> .
<code>inline-block</code>	O melhor dos dois mundos: fica lado a lado como <code>inline</code> , mas aceita <code>width</code> , <code>height</code> e <code>margin/padding</code> completos como <code>block</code> .
<code>none</code>	Remove o elemento completamente da renderização (some da tela e não ocupa espaço).
<code>flex</code>	Transforma o elemento em um container flexível (veja Módulo 11).
<code>grid</code>	Transforma o elemento em um container de grade (veja Módulo 12).

⚠ **display: none vs visibility: hidden**

`display: none` remove o elemento do fluxo — o espaço dele desaparece. Já `visibility: hidden` apenas o torna invisível, mas **mantém o espaço** reservado na página. São efeitos bem diferentes visualmente.

A propriedade position

Valor	Comportamento
<code>static</code>	Padrão de todos os elementos. Segue o fluxo normal da página, e <code>top/left/right/bottom</code> não têm efeito.
<code>relative</code>	Segue o fluxo normal, mas permite deslocar o elemento a partir de sua posição original usando <code>top/left/right/bottom</code> , sem afetar os outros elementos.

<code>absolute</code>	Sai completamente do fluxo normal e se posiciona em relação ao ancestral mais próximo que tenha <code>position</code> diferente de static.
<code>fixed</code>	Sai do fluxo e se fixa em relação à janela do navegador (viewport) — não se move nem com o scroll da página.
<code>sticky</code>	Comporta-se como relative até um certo ponto do scroll, quando então "gruda" (como fixed) dentro do seu container pai.

EXEMPLO CLÁSSICO: ABSOLUTE DENTRO DE RELATIVE

```
.card {
  position: relative;
}

.selo {
  position: absolute;
  top: 10px;
  right: 10px;
}
```

Resultado esperado

O elemento com classe "selo" (ex: uma etiqueta "Promoção") ficará "grudado" no canto superior direito do ".card", mesmo que o card mude de tamanho ou posição na página. Isso funciona porque o ".card" virou a "referência" de posicionamento ao receber `position: relative`.

EXEMPLO — MENU FIXO NO TOPO

```
.menu-topo {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  background: white;
  z-index: 1000;
}
```

z-index — controlando a ordem de empilhamento

Quando elementos se sobrepõem, o `z-index` decide quem fica "por cima". Só funciona em elementos que tenham um `position` diferente de `static`. Quanto maior o valor, mais "para frente" o elemento fica.

Overflow — controlando conteúdo que "vaza"

Valor	Comportamento
<code>visible</code>	Padrão. O conteúdo que ultrapassa a caixa continua visível, "vazando" para fora.
<code>hidden</code>	Esconde qualquer conteúdo que ultrapasse os limites da caixa.
<code>scroll</code>	Sempre adiciona barras de rolagem, mesmo que não sejam necessárias.
<code>auto</code>	Adiciona barra de rolagem apenas quando necessário.



Exercício de fixação — Módulo 10

1. Crie um card com um "selo" no canto usando `position relative + absolute`.
2. Crie um menu que fique fixo no topo da página mesmo durante o scroll.
3. Explique com suas palavras a diferença entre `display:none` e `visibility:hidden`.

Flexbox Completo

O que é Flexbox e por que ele existe

Antes do Flexbox, centralizar uma caixa verticalmente em CSS era motivo de piada entre desenvolvedores — exigia truques complicados. O Flexbox foi criado para resolver, de forma simples, o problema de **distribuir e alinhar elementos** dentro de um container, seja em linha ou em coluna.

Os dois eixos do Flexbox

Para entender Flexbox, você precisa fixar um conceito: todo container flex tem um **eixo principal** (main axis) e um **eixo cruzado** (cross axis).

- Se `flex-direction: row` (padrão) → eixo principal é **horizontal**, eixo cruzado é **vertical**.
- Se `flex-direction: column` → eixo principal é **vertical**, eixo cruzado é **horizontal**.



Por que isso importa tanto

A maioria das propriedades do Flexbox depende de saber qual é o eixo principal no momento. `justify-content` sempre alinha no eixo **principal**, e `align-items` sempre alinha no eixo **cruzado**. Decore essa regra e o Flexbox nunca mais vai te confundir.

Ativando o Flexbox

```
.container {  
  display: flex;  
}
```

Basta essa linha para transformar todos os filhos diretos desse elemento em "itens flexíveis", que automaticamente se organizam lado a lado.

Propriedades do container (o "pai")

Propriedade	Valores	Efeito
<code>flex-direction</code>	row / row-reverse / column / column-reverse	Define a direção do eixo principal
<code>justify-content</code>	flex-start / center / flex-end / space-between / space-around / space-evenly	Alinha os itens no eixo principal
<code>align-items</code>	flex-start / center / flex-end / stretch / baseline	Alinha os itens no eixo cruzado
<code>flex-wrap</code>	nowrap / wrap / wrap-reverse	Permite (ou não) que os itens quebrem para a próxima linha quando não cabem
<code>gap</code>	ex: 16px	Espaçamento entre os itens, sem precisar de margin manual
<code>align-content</code>	igual ao align-items	Alinha múltiplas linhas de itens (só faz efeito com flex-wrap: wrap)

EXEMPLO — CENTRALIZAR TUDO (O "SANTO GRAAL" DO CSS)

```
.container {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```



Resultado esperado

Qualquer elemento filho dessa div ficará perfeitamente centralizado, tanto na horizontal quanto na vertical, ocupando a altura inteira da tela. Esse é provavelmente o trecho de CSS mais usado no mundo.

Propriedades dos itens (os "filhos")

Propriedade	Efeito
-------------	--------

<code>flex-grow</code>	Define o quanto o item pode crescer para ocupar espaço livre (0 = não cresce, padrão)
<code>flex-shrink</code>	Define o quanto o item pode encolher quando falta espaço (1 = encolhe, padrão)
<code>flex-basis</code>	Tamanho inicial do item antes de crescer ou encolher
<code>flex</code>	Atalho para grow, shrink e basis de uma vez, ex: <code>flex: 1;</code>
<code>align-self</code>	Sobrescreve o align-items apenas para este item específico
<code>order</code>	Muda a ordem visual do item sem alterar o HTML (padrão: 0)

EXEMPLO — TRÊS COLUNAS DE LARGURAS PROPORCIONAIS

```
.coluna-menor { flex: 1; }
.coluna-maior { flex: 2; }
```

Aqui, a coluna-maior sempre ocupará o dobro do espaço da coluna-menor, não importa o tamanho da tela — essa é a base de layouts fluidos e responsivos.

Exemplo completo: barra de navegação com Flexbox

HTML

```
<nav class="navbar">
  <div class="logo">MinhaMarca</div>
  <ul class="links">
    <li><a href="#">Início</a></li>
    <li><a href="#">Sobre</a></li>
    <li><a href="#">Contato</a></li>
  </ul>
</nav>
```

CSS

```
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 16px 32px;  
}  
  
.links {  
  display: flex;  
  gap: 24px;  
  list-style: none;  
}
```



Resultado esperado

O logo fica alinhado à esquerda e os links do menu à direita (graças ao `space-between`), todos perfeitamente alinhados na vertical (graças ao `align-items: center`), com espaçamento uniforme de 24px entre os links.



Exercício de fixação — Módulo 11

1. Crie um container flex com 3 caixas coloridas e distribua-as com `justify-content: space-around`.
2. Centralize um texto na tela inteira (horizontal e vertical) usando Flexbox.
3. Crie uma barra de navegação com logo à esquerda e menu à direita.

CSS Grid Completo

Grid vs Flexbox — quando usar cada um

💡 Regra prática

Flexbox foi feito para organizar elementos em **uma dimensão** (uma linha OU uma coluna por vez). Grid foi feito para organizar em **duas dimensões** (linhas E colunas simultaneamente). Use Flexbox para menus, barras e alinhamentos simples. Use Grid para o layout geral da página (cabeçalho, sidebar, conteúdo, rodapé) e para grades de cards/produtos.

Ativando o Grid

```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
  gap: 16px;  
}
```

Isso cria uma grade com 3 colunas fixas de 200px cada, e 16px de espaçamento entre todas as células.

A unidade fr (fração)

A unidade `fr` representa uma "fração" do espaço disponível — é o que torna o Grid tão poderoso para layouts fluidos:

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr;  
}
```

Aqui a coluna do meio sempre ocupará o dobro do espaço das colunas laterais, adaptando-se automaticamente ao tamanho da tela.

repeat() e minmax() — grades inteligentes

```
.galeria {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 20px;  
}
```

`repeat(3, 1fr)` é apenas um atalho para escrever `1fr 1fr 1fr`.

```
.galeria-responsiva {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));  
  gap: 20px;  
}
```

Resultado esperado

Esta é uma das combinações mais úteis do CSS Grid: ela cria automaticamente quantas colunas couberem na tela, garantindo que cada uma tenha no mínimo 200px de largura. Em uma tela grande, aparecem mais colunas; em um celular, aparece apenas 1 coluna — **tudo sem precisar de nenhuma media query!**

Linhas (rows)

```
.container {  
  grid-template-rows: 100px auto 80px;  
}
```

grid-template-areas — a forma mais visual de criar layouts

Permite "desenhar" o layout da página literalmente com nomes, direto no CSS:

HTML

```
<div class="pagina">
  <header>Cabeçalho</header>
  <nav>Menu</nav>
  <main>Conteúdo</main>
  <footer>Rodapé</footer>
</div>
```

CSS

```
.pagina {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-areas:
    "cabecalho cabecalho"
    "menu conteudo"
    "rodape rodape";
  gap: 10px;
}

header { grid-area: cabecalho; }
nav     { grid-area: menu; }
main    { grid-area: conteudo; }
footer  { grid-area: rodape; }
```

Resultado esperado

O cabeçalho e o rodapé ocupam a largura total (aparecem duas vezes seguidas no template, uma para cada coluna). O menu fica em uma coluna estreita de 200px à esquerda, e o conteúdo ocupa o restante do espaço. Basta olhar para o `grid-template-areas` para "visualizar" o layout inteiro.

Fazendo um item ocupar várias células

```
.item-grande {
  grid-column: span 2; /* ocupa 2 colunas */
  grid-row: span 2;    /* ocupa 2 linhas */
}
```



Exercício de fixação — Módulo 12

1. Crie uma galeria de 6 cards usando `repeat(auto-fit, minmax(150px, 1fr))`.
2. Monte o layout clássico header/sidebar/main/footer usando `grid-template-areas`.
3. Explique com suas palavras quando usar Grid em vez de Flexbox.

Responsividade

O que é design responsivo?

É a prática de construir sites que se adaptam automaticamente a qualquer tamanho de tela — celular, tablet, notebook ou monitor grande — oferecendo uma boa experiência em todos os casos, sem a necessidade de "sites separados" para mobile.

Mobile-first: comece pelo menor

A abordagem recomendada pelo mercado é escrever o CSS **pensando primeiro no celular** (a tela mais restrita) e depois ir "adicionando" estilos para telas maiores usando media queries. Isso, geralmente, resulta em um CSS mais enxuto e organizado.

A meta tag viewport (pré-requisito)

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Sem essa linha no `<head>`, nenhuma media query funcionará corretamente em dispositivos móveis — o navegador vai simular uma tela "desktop" reduzida ao invés de usar a largura real do aparelho.

Media Queries na prática

```

/* Estilo padrão: aplicado a TODAS as telas (mobile-first) */
.container {
  display: flex;
  flex-direction: column;
}

/* A partir de 768px (tablets) */
@media (min-width: 768px) {
  .container {
    flex-direction: row;
  }
}

/* A partir de 1024px (desktops) */
@media (min-width: 1024px) {
  .container {
    max-width: 1200px;
    margin: 0 auto;
  }
}

```

No exemplo acima: em celulares, os itens ficam empilhados (coluna). A partir de 768px de largura de tela (tablets), eles passam a ficar lado a lado (linha). A partir de 1024px (desktop), o container ganha uma largura máxima e é centralizado.

Breakpoints mais usados no mercado

Faixa	Dispositivo aproximado
até 480px	Celulares pequenos
481px – 768px	Celulares grandes / tablets em pé
769px – 1024px	Tablets deitados / notebooks pequenos
1025px em diante	Desktops e monitores grandes

✓ Dica

Esses valores são apenas referências. O ideal é definir breakpoints com base em quando **o seu próprio layout** começa a "quebrar" visualmente — abra o DevTools do navegador (F12), ative o modo responsivo e vá redimensionando até encontrar o ponto exato onde o design não funciona mais bem.

Imagens e vídeos responsivos

```
img, video {  
  max-width: 100%;  
  height: auto;  
}
```

Essa regra simples garante que imagens e vídeos nunca "estourem" a largura do container onde estão, encolhendo proporcionalmente em telas menores.



Exercício de fixação — Módulo 13

1. Crie um layout de 3 colunas que vira 1 coluna em telas menores que 768px.
2. Explique a diferença entre a abordagem mobile-first e desktop-first.
3. Teste seu site no DevTools em pelo menos 3 tamanhos de tela diferentes.

Efeitos Visuais e Animações

Hover e Focus

```
.botao {
  background-color: #2563eb;
  color: white;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
}

.botao:hover {
  background-color: #1d4ed8;
}
```

Transições — a chave para animações suaves

Sem a propriedade `transition`, qualquer mudança de estilo (como no hover acima) acontece de forma instantânea e "seca". A transição suaviza essa troca ao longo de um tempo definido.

```
.botao {
  background-color: #2563eb;
  transition: background-color 0.3s ease;
}

.botao:hover {
  background-color: #1d4ed8;
}
```

Parte da sintaxe	Significado
<code>background-color</code>	Qual propriedade será animada (pode usar <code>all</code> para todas)
<code>0.3s</code>	Duração da transição

<code>ease</code>	Curva de aceleração: ease (padrão suave), linear (velocidade constante), ease-in, ease-out, ease-in-out
-------------------	---

Transform — movendo, girando e escalando elementos

Função	Efeito
<code>translate(x, y)</code>	Move o elemento sem afetar o fluxo dos outros elementos
<code>scale(n)</code>	Aumenta ou diminui o tamanho (1 = normal, 1.1 = 10% maior)
<code>rotate(deg)</code>	Gira o elemento em graus
<code>skew(deg)</code>	Inclina o elemento

EXEMPLO — CARD QUE CRESCE E "SOBE" NO HOVER

```
.card {
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
  transform: translateY(-8px) scale(1.02);
  box-shadow: 0 12px 24px rgba(0,0,0,0.15);
}
```



Resultado esperado

Ao passar o mouse, o card "flutua" suavemente para cima 8 pixels, cresce 2% e ganha uma sombra mais forte — um efeito muito comum em sites modernos de produtos e portfólios.

Animações com @keyframes

Enquanto `transition` anima entre 2 estados (normal → hover), o `@keyframes` permite criar animações com **múltiplas etapas**, que rodam automaticamente, em loop se quiser.

```
@keyframes pulsar {
  0%   { transform: scale(1); }
  50%  { transform: scale(1.1); }
  100% { transform: scale(1); }
}

.notificacao {
  animation: pulsar 1.5s ease-in-out infinite;
}
```

Parte da sintaxe	Significado
pulsar	Nome da animação (deve casar com o nome usado em @keyframes)
1.5s	Duração de um ciclo completo
ease-in-out	Curva de aceleração
infinite	Repete para sempre (poderia ser um número, ex: 3)

Sombras

```
.caixa {
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

.texto {
  text-shadow: 1px 1px 2px rgba(0,0,0,0.3);
}
```

A sintaxe do `box-shadow` segue: deslocamento horizontal, deslocamento vertical, desfoque (blur), e cor. Quanto maior o desfoque, mais "suave" a sombra.

Gradientes

```
.banner {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
```



Exercício de fixação — Módulo 14

1. Crie um botão com transição suave de cor de fundo no hover.
2. Crie uma animação de "pulsar" usando @keyframes em um ícone de notificação.
3. Aplique um box-shadow e um gradiente de fundo em um card.

Variáveis CSS, Boas Práticas e Projeto Final

Variáveis CSS (Custom Properties)

Permitem guardar valores reutilizáveis (cores, espaçamentos, fontes) em um único lugar, facilitando manutenção — se você mudar a cor principal do site, muda em um só lugar, ao invés de em dezenas de regras.

```
:root {
  --cor-primaria: #2563eb;
  --cor-texto: #1a1a1a;
  --espacamento: 16px;
  --fonte-principal: 'Segoe UI', sans-serif;
}

.botao {
  background-color: var(--cor-primaria);
  padding: var(--espacamento);
  font-family: var(--fonte-principal);
}
```

`:root` representa o elemento raiz do documento (equivalente ao `<html>`), e é o local padrão para declarar variáveis globais.

Boas práticas de organização

- **Nomeie classes de forma descritiva:** prefira `.card-produto` a `.div1`.
- **Evite IDs para estilização:** reserve-os para JavaScript e âncoras.
- **Evite !important:** é sinal de que a especificidade do CSS está desorganizada.
- **Separe o CSS em seções comentadas:** reset, variáveis, layout, componentes, responsividade.
- **Use box-sizing: border-box globalmente.**
- **Priorize classes reutilizáveis** a estilos únicos e repetidos.

Projeto Prático Final: Landing Page

Vamos aplicar tudo que foi visto nesta apostila em uma landing page simples, com cabeçalho, seção de destaque (hero), cards de serviços e rodapé — totalmente responsiva.

HTML (INDEX.HTML)

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MinhaMarca - Soluções Digitais</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="navbar">
    <div class="logo">MinhaMarca</div>
    <nav>
      <ul class="links">
        <li><a href="#servicos">Serviços</a></li>
        <li><a href="#contato">Contato</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section class="hero">
      <h1>Transforme sua ideia em realidade</h1>
      <p>Criamos sites rápidos, bonitos e responsivos para o seu negócio.</p>
      <a href="#contato" class="botao">Fale conosco</a>
    </section>

    <section id="servicos" class="servicos">
      <h2>Nossos Serviços</h2>
      <div class="grid-servicos">
        <article class="card">
          <h3>Desenvolvimento Web</h3>
          <p>Sites e sistemas sob medida.</p>
        </article>
        <article class="card">
          <h3>Design UI/UX</h3>
          <p>Interfaces bonitas e intuitivas.</p>
        </article>
        <article class="card">
          <h3>SEO</h3>
          <p>Mais visibilidade no Google.</p>
        </article>
      </div>
    </section>
  </main>
</body>
</html>
```

```
    </section>
</main>

<footer id="contato">
  <p>&copy; 2026 MinhaMarca. Todos os direitos reservados.</p>
</footer>

</body>
</html>
```

CSS (STYLE.CSS)

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

:root {
  --cor-primaria: #2563eb;
  --cor-texto: #1a1a1a;
}

body {
  font-family: 'Segoe UI', Arial, sans-serif;
  color: var(--cor-texto);
  line-height: 1.6;
}

/* Navbar */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px 40px;
}

.links {
  display: flex;
  gap: 24px;
  list-style: none;
}

.links a {
  text-decoration: none;
  color: var(--cor-texto);
}

/* Hero */
.hero {
  text-align: center;
  padding: 80px 20px;
  background: linear-gradient(135deg, #eff6ff, #f5f3ff);
}

.hero h1 {
```

```
    font-size: 2.5rem;
    margin-bottom: 16px;
}

.botao {
    display: inline-block;
    margin-top: 24px;
    padding: 14px 32px;
    background-color: var(--cor-primaria);
    color: white;
    text-decoration: none;
    border-radius: 6px;
    transition: background-color 0.3s ease, transform 0.3s ease;
}

.botao:hover {
    background-color: #1d4ed8;
    transform: translateY(-3px);
}

/* Serviços */
.servicos {
    padding: 60px 40px;
    text-align: center;
}

.grid-servicos {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 24px;
    margin-top: 32px;
}

.card {
    background: white;
    padding: 32px 20px;
    border-radius: 10px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.08);
    transition: transform 0.3s ease;
}

.card:hover {
    transform: translateY(-6px);
}
```

```

/* Footer */
footer {
  text-align: center;
  padding: 24px;
  background: #0f172a;
  color: white;
}

/* Responsividade */
@media (max-width: 600px) {
  .navbar {
    flex-direction: column;
    gap: 12px;
  }
  .hero h1 {
    font-size: 1.8rem;
  }
}

```



Resultado esperado

Uma landing page completa: menu com logo e links (Flexbox), seção de destaque centralizada com gradiente de fundo e botão animado, uma grade de 3 cards de serviços que se reorganiza automaticamente em telas pequenas (Grid + minmax), efeitos de hover suaves em botão e cards, e um rodapé escuro. Em telas menores que 600px, o menu empilha verticalmente e o título principal diminui — tudo isso com apenas uma media query.



Desafio final

1. Reproduza esta landing page em seu editor de código e teste no navegador.
2. Troque as cores usando apenas as variáveis CSS em :root.
3. Adicione uma nova seção "Depoimentos" usando Flexbox ou Grid, com pelo menos 2 cards.
4. Torne o menu responsivo, ocultando os links em telas pequenas (dica: pesquise sobre "menu hambúrguer" como próximo passo, já usando JavaScript).

Parabéns por chegar até aqui! 🎉

Você agora tem uma base sólida e completa de HTML5 e CSS3. O próximo passo natural é aprender **JavaScript** para dar interatividade aos seus projetos. Continue praticando — a prática constante é o que realmente transforma teoria em habilidade profissional.